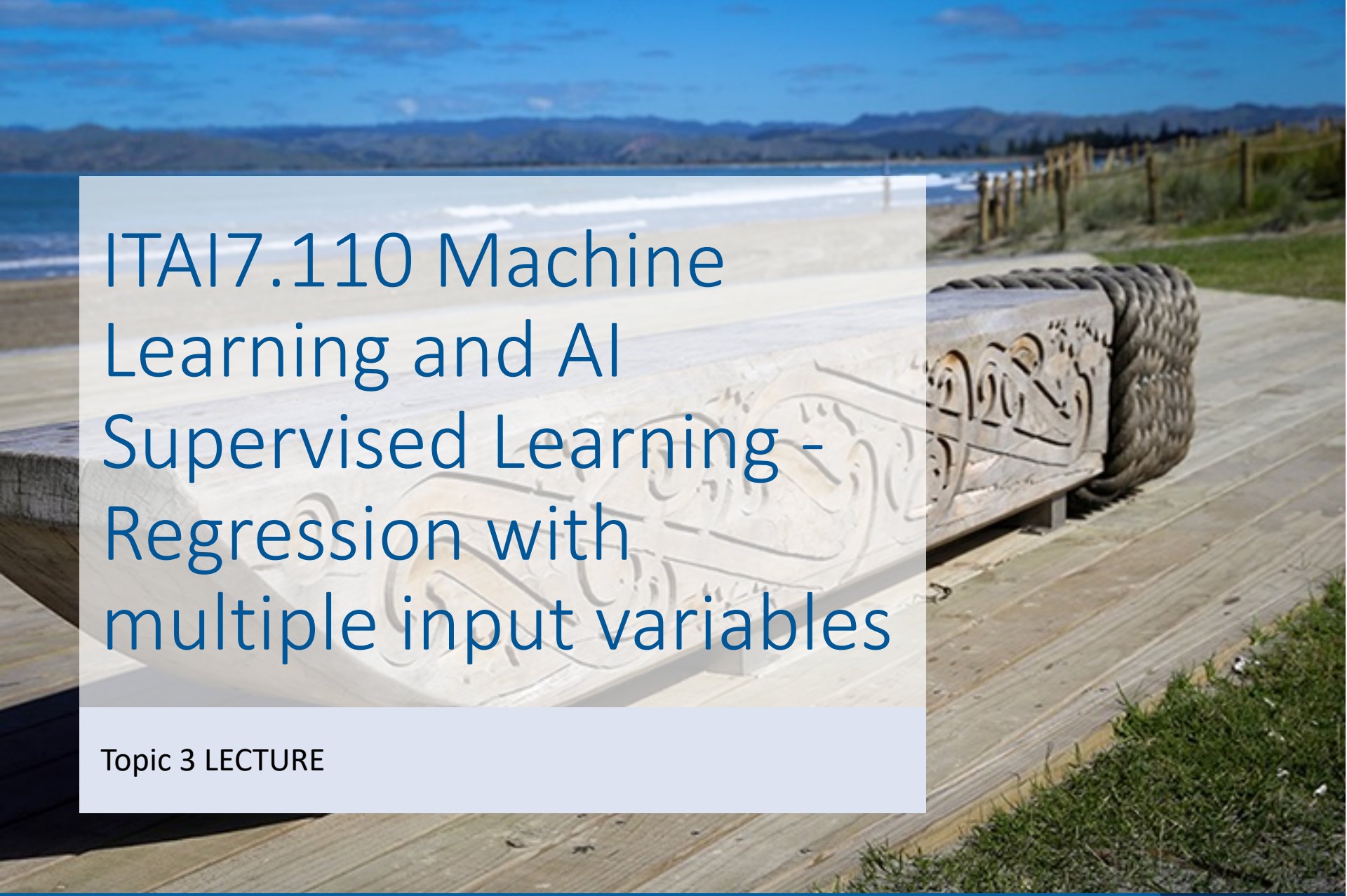


ITAI7.110 Machine Learning and AI Supervised Learning - Regression with multiple input variables

Topic 3 LECTURE

Review for Week 2

- Data Pre-processing Techniques
- Linear regression(线性回归)
- Logistic regression(逻辑回归)

Outline

- Multiple Linear Regression(多元线性回归)
- Polynomial Regression(多项式回归)
- Evaluating a Model
- Regularisation(正则化) techniques: L1, L2

Multiple Regression
&
Polynomial Regression

Introduction to Multiple Linear Regression

- Linear regression is useful when we want to predict the values of a variable from its relationship with other variables.

- There are two different types of linear regression models.
  - simple linear regression
  - multiple linear regression

- In predicting the price of a home, one factor to consider is the size of the home. The relationship between those two variables, price and size, is important, but there are other variables:
  - location, air quality, demographics, parking, and more.
- When we want to use multiple independent variables, we'll use Multiple Linear Regression.

Multiple Linear Regression

- uses two or more independent variables to predict the values of the dependent variable. It is based on the following equation that we'll explore later on:

$$y = b + m_1 x_1 + m_2 x_2 + \ldots + m_n x_n$$

Multiple Linear Regression Equation

- Equation 1： The equation for multiple linear regression that uses two independent variables is this:

$$y = b + m_1 x_1 + m_2 x_2$$

- Equation 2： The equation for multiple linear regression that uses three independent variables is this:

$$y = b + m_1 x_1 + m_2 x_2 + m_3 x_3$$

- Equation 3： As a result, since multiple linear regression can use any number of independent variables, its general equation becomes:

$$y = b + m_1 x_1 + m_2 x_2 + \ldots + m_n x_n$$

- Here, m_1, m_2, m_3, … m_n refer to the coefficients, and b refers to the intercept.

- Coefficients are most helpful in determining which independent variable carries more weight. For example, a coefficient of -1.345 will impact the rent more than a coefficient of 0.238, with the former impacting prices negatively and latter positively.

Correlations

- In the house price model, we used 14 variables, so there are 14 coefficients:

```
[ -302.73009383  1199.3859951  4.79976742
 -24.28993151  24.19824177 -7.58272473
-140.90664773  48.85017415 191.4257324
-151.11453388  89.408889  -57.89714551
-19.31948556  -38.92369828 ]
```

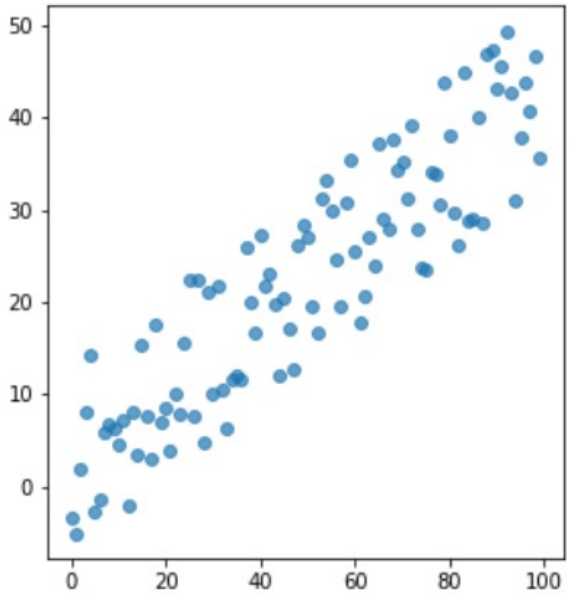
- In regression, the independent variables will either have a positive linear relationship to the dependent variable, a negative linear relationship, or no relationship.

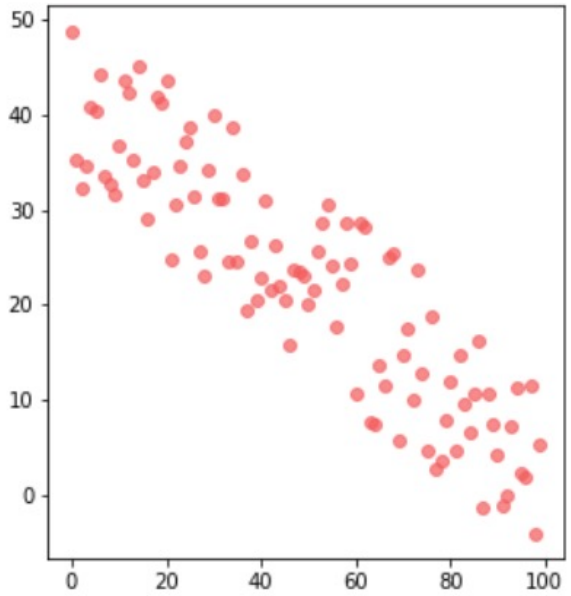
- A negative linear relationship means that as X values increase, Y values will decrease.

- Similarly, a positive linear relationship means that as X values increase, Y values will also increase.

- Graphically, when you see a downward trend, it means a negative linear relationship exists. When you find an upward trend, it indicates a positive linear relationship.

- Here are two graphs indicating positive and negative linear relationships:





Review

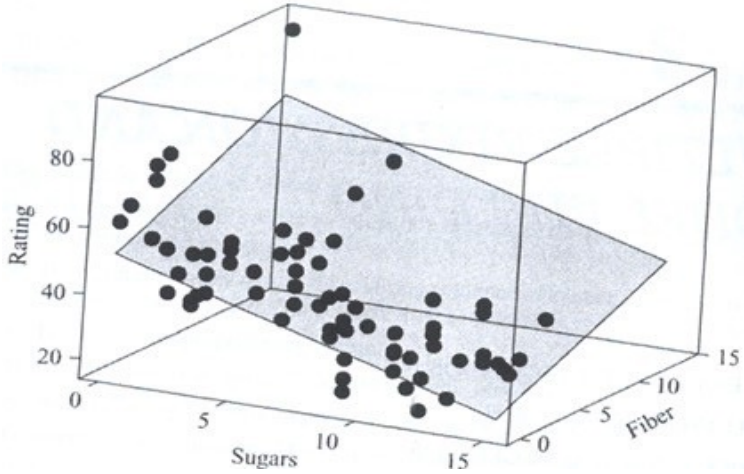
- Multiple Linear Regression uses two or more variables to make predictions about another variable:

$$y = b + m_1x_1 + m_2x_2 + \ldots + m_nx_n$$

- Multiple linear regression uses a set of independent variables and a dependent variable. It uses these variables to learn how to find optimal parameters.

Multiple regression

- Simple regression: 1 feature
- Multiple regression: more than 1 feature



- The line becomes a plane in 2-dim. space and a hyperplane in >2-dim. space
- R2 is similarly defined, called multiple coefficient of determination

R^2 of the prediction

- The coefficient R^2 is defined as:

$$1 - \frac{u}{v}$$

- where u is the residual sum of squares:

```
((y - y_predict) ** 2).sum()
```

- and v is the total sum of squares (TSS):

```
((y - y.mean()) ** 2).sum()
```

- R^2 is the percentage variation in y explained by all the x variables together.

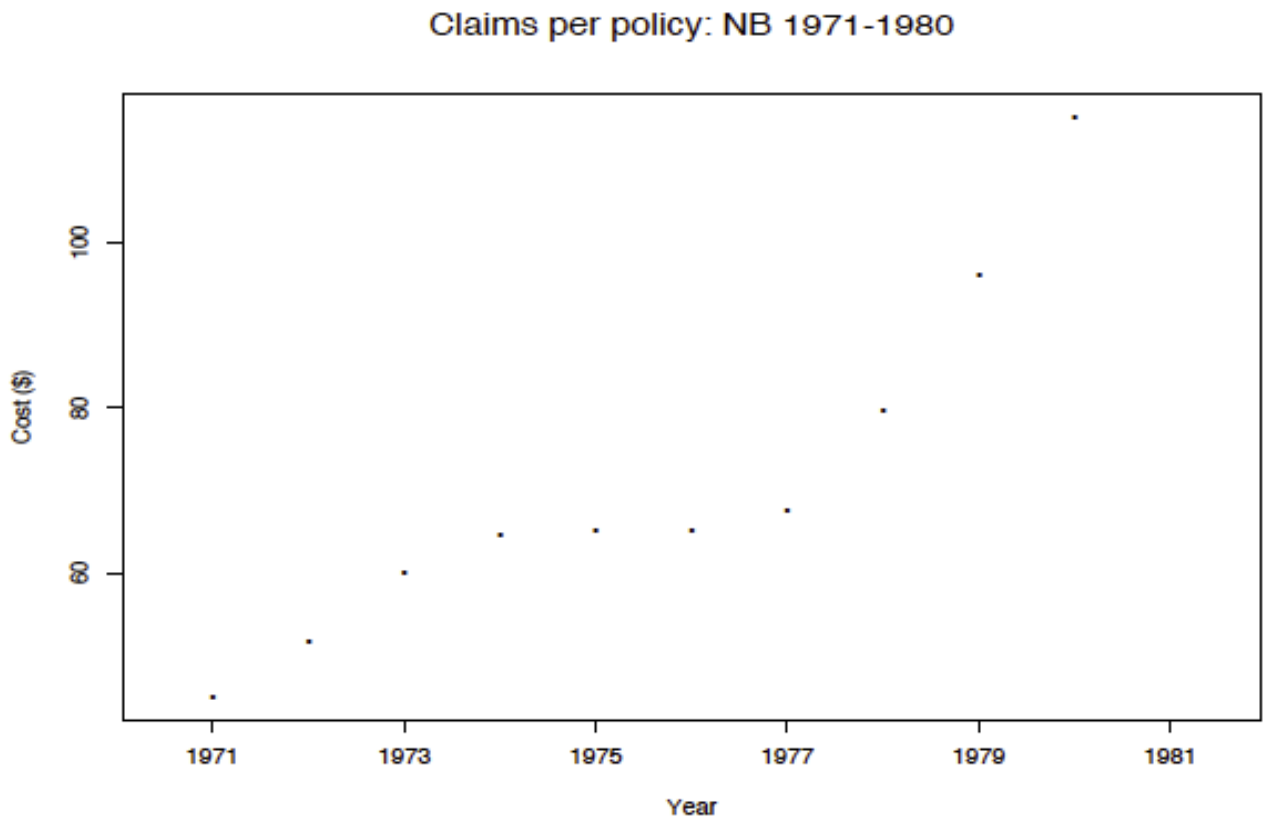
- The best possible R^2 is 1.00 (and it can be negative because the model can be arbitrarily worse). Usually, a R^2 of 0.70 is considered good.

Polynomial Regression

- Data: average claims paid per policy for automobile insurance in New Brunswick in the years 1971-1980:

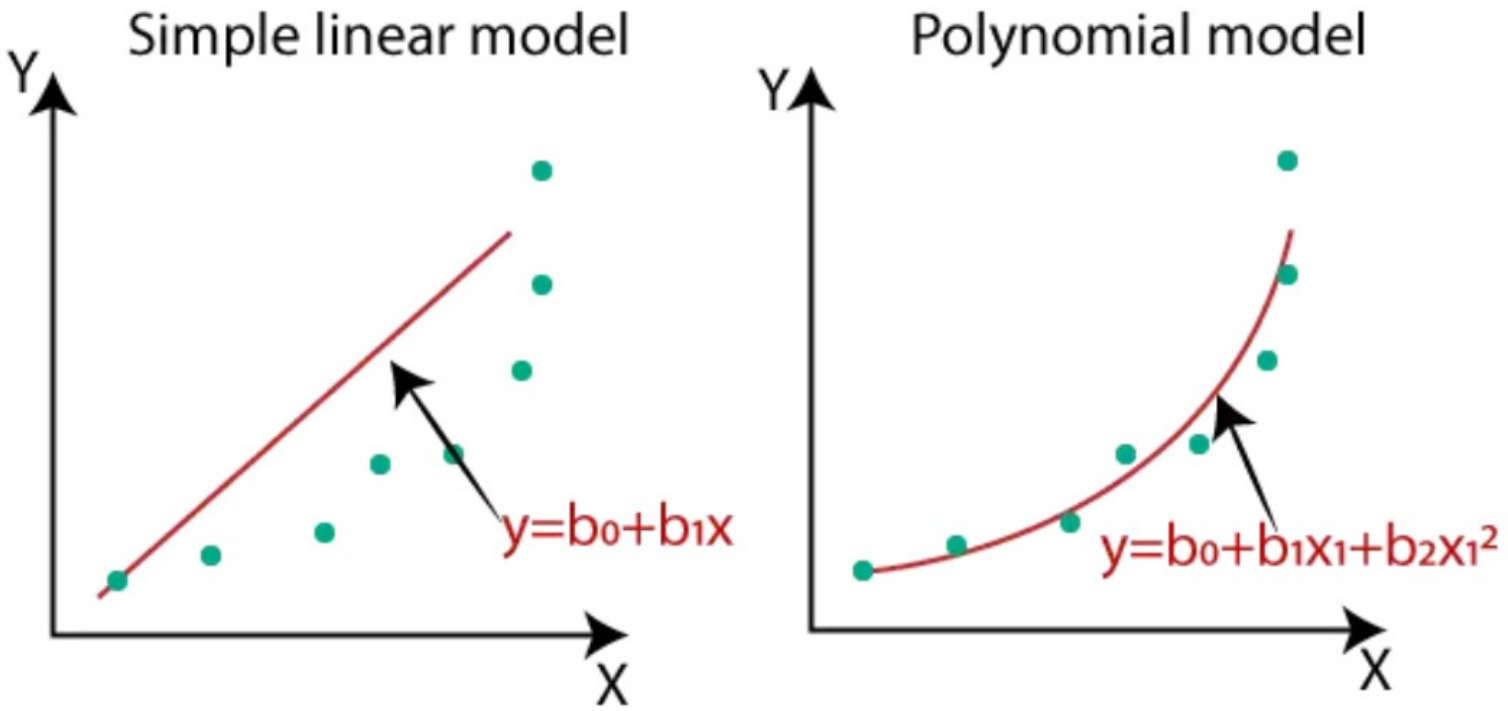
Year	1971	1972	1973	1974	1975
Cost	45.13	51.71	60.17	64.83	65.24
Year	1976	1977	1978	1979	1980
Cost	65.17	67.65	79.80	96.13	115.19

Data Plot



EIT TE AHO A MĀUI | TePūkenga

Polynomial Regression



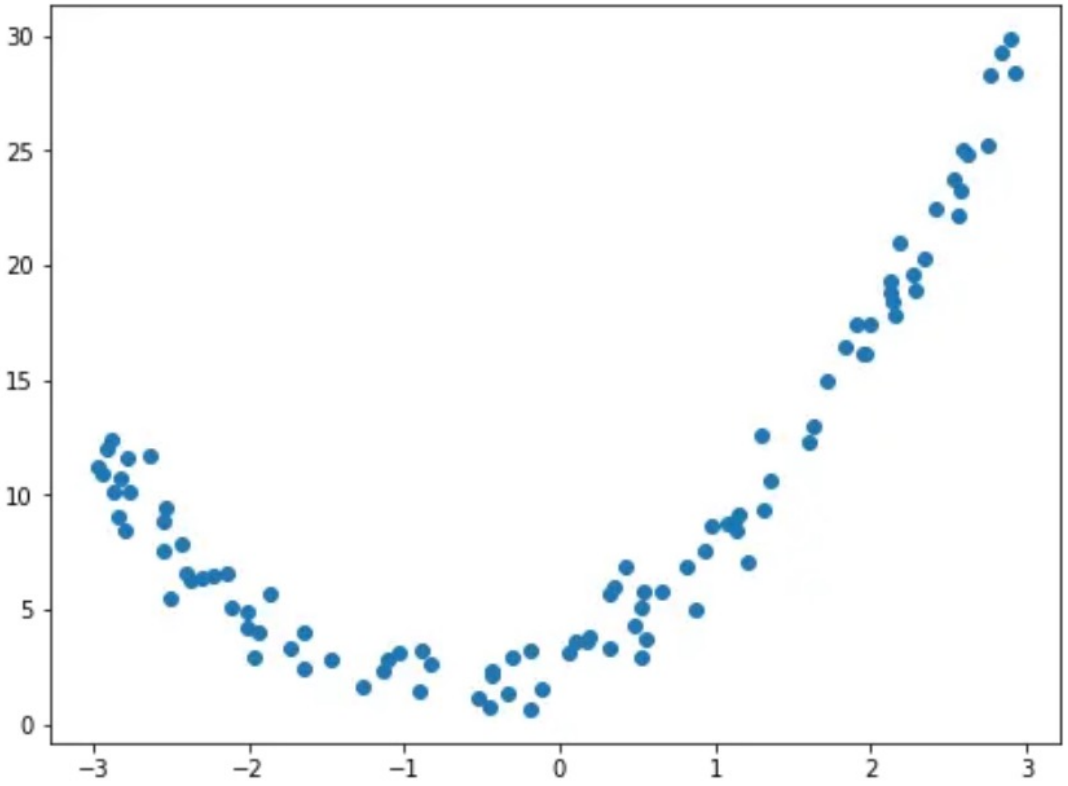
Polynomial Regression

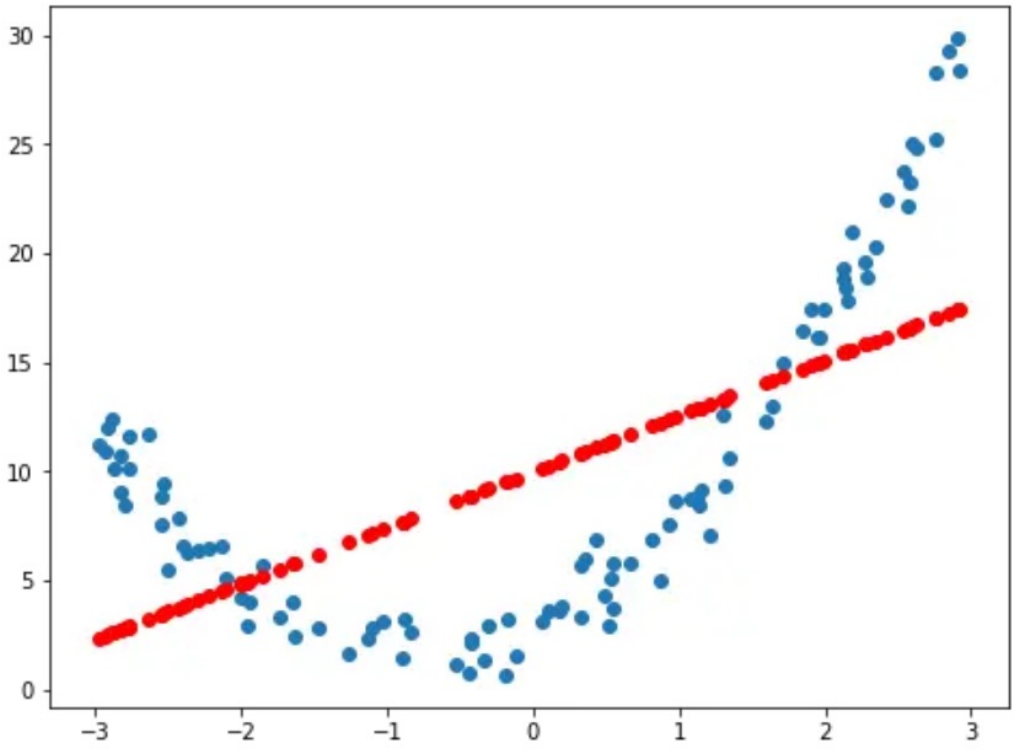
- Polynomial models are very powerful to handle nonlinearity, because polynomials can approximate continuous functions within any given precision

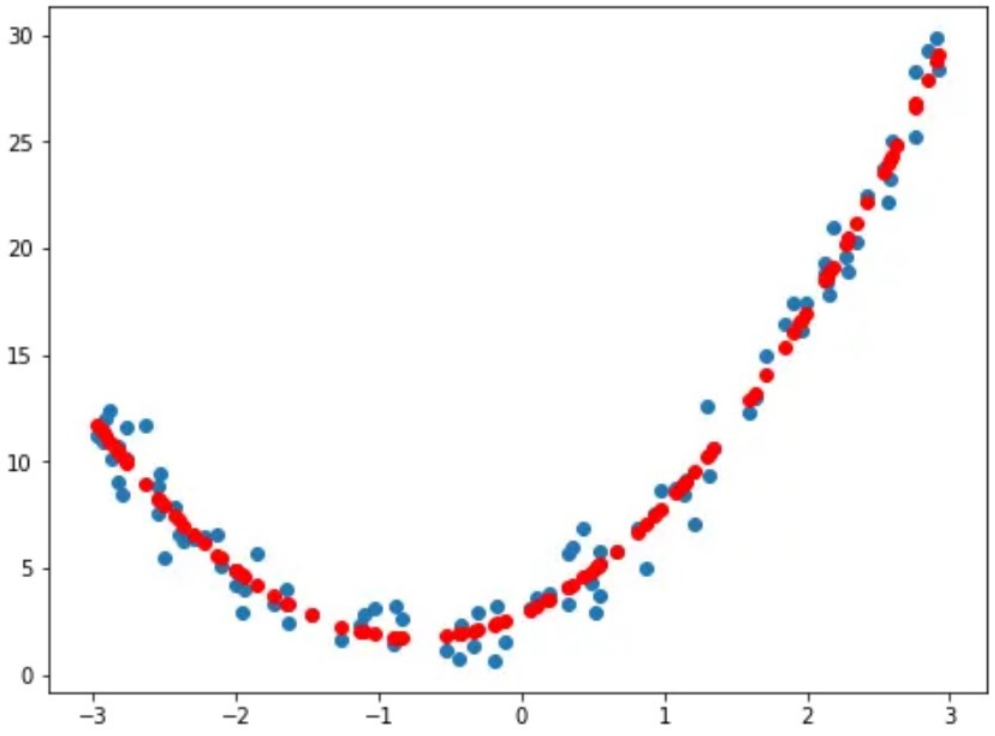
$$y = \beta_0 + \beta_1 \underbrace{x}_{x_1} + \cdots + \beta_k \underbrace{x^k}_{x_k} + \epsilon$$

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \underbrace{\beta_{11} x_1^2}_{\beta_3 x_3} + \underbrace{\beta_{22} x_2^2}_{\beta_4 x_4} + \underbrace{\beta_{12} x_1 x_2}_{\beta_5 x_5} + \epsilon$$

- However, such fitting problems can still be treated as linear regression:


30
25
20
15
10
5
0
−3
−2
−1
0
1
2
3


30
25
20
15
10
5
0
−3
−2
−1
0
1
2
3


30
25
20
15
10
5
0
−3
−2
−1
0
1
2
3

Model Selection and Evaluation

Model Evaluation

- Confusion Matrix
- Precision, Recall, and F1 Score
- Balanced Accuracy
- ROC
- Extending Binary Metrics to Multi-class Settings

Confusion matrix

- A confusion matrix, which shows the number of true positives, false positives, true negatives, and false negatives.

- For example, suppose that the true and predicted classes for a logistic regression model are:

```
y_true = [0, 0, 1, 1, 1, 0, 0, 1, 0, 1]
y_pred = [0, 1, 1, 0, 1, 0, 1, 1, 0, 1]
```

- TN=3, FP=2, FN=1, TP=4.

- We can create a confusion matrix as follows:

```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_true, y_pred))
```

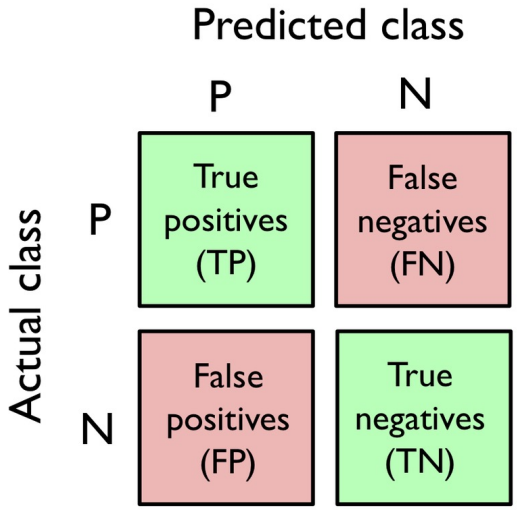
- Output:

```
array([[3, 2],
       [1, 4]])
```

- Ideally, we want the numbers on the main diagonal to be as large as possible.

```
array([[3, 2],
       [1, 4]])
```

Confusion Matrix



$$ERR = \frac{FP + FN}{FP + FN + TP + TN} = 1 - ACC$$

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} = 1 - ERR$$

False Positive Rate & False Negative Rate

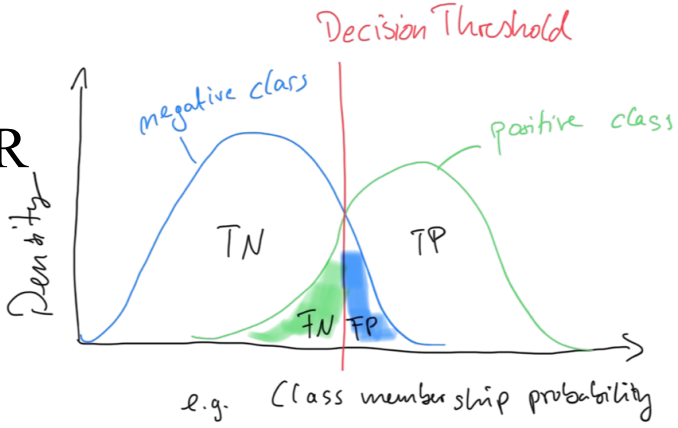
$$TPR^{*} = \frac{TP}{P} = \frac{TP}{TP + FN} = 1 - FNR$$

$$FPR^{*} = \frac{FP}{N} = \frac{FP}{FP + TN} = 1 - TNR$$

$$FNR = \frac{FN}{P} = \frac{FN}{FN + TP} = 1 - TPR$$

$$TNR = \frac{TN}{N} = \frac{TN}{TN + FP} = 1 - FPR$$

* Relevant later for ROC



- Think of it in a spam classification problem (what are true positives, and if you had to pick one at the expense of the other: would you rather decrease the FPR or increase the TPR?)

EIT TE AHO A MĀUI | TePūkenga

- Once we have a confusion matrix, there are a few different statistics we can use to summarize the four values in the matrix. These include accuracy, precision, recall, and F1 score.

Accuracy

- The simplest way of reporting the effectiveness of an algorithm is by calculating its accuracy. Accuracy is calculated by finding the total number of correctly classified points and dividing by the total number of points.

- In other words, accuracy can be defined as:

$$\frac{True\ Positives + True\ Negatives}{True\ Positives + True\ Negatives + False\ Positives + FalseNegatives}$$

Recall

- Recall measures the percentage of relevant items that your classifier found.

$$\frac{True\ Positives}{True\ Positives + False\ Negatives}$$

Precision

- The formula for precision is below:

$$\frac{True\ Positives}{True\ Positives + False\ Positives}$$

- Precision and recall are statistics that are on opposite ends of a scale. If one goes down, the other will go up.

F1 Score

- F1 score is the harmonic mean of precision and recall. The harmonic mean of a group of numbers is a way to average them together. The formula for F1 score is below:

$$2 * \frac{Precision * Recall}{Precision + Recall}$$

Review

- Classifying a single point can result in a true positive, a true negative, a false positive, or a false negative.
- Accuracy measures how many classifications your algorithm got correct out of every classification it made.
- Recall measures the percentage of the relevant items your classifier was able to successfully find.

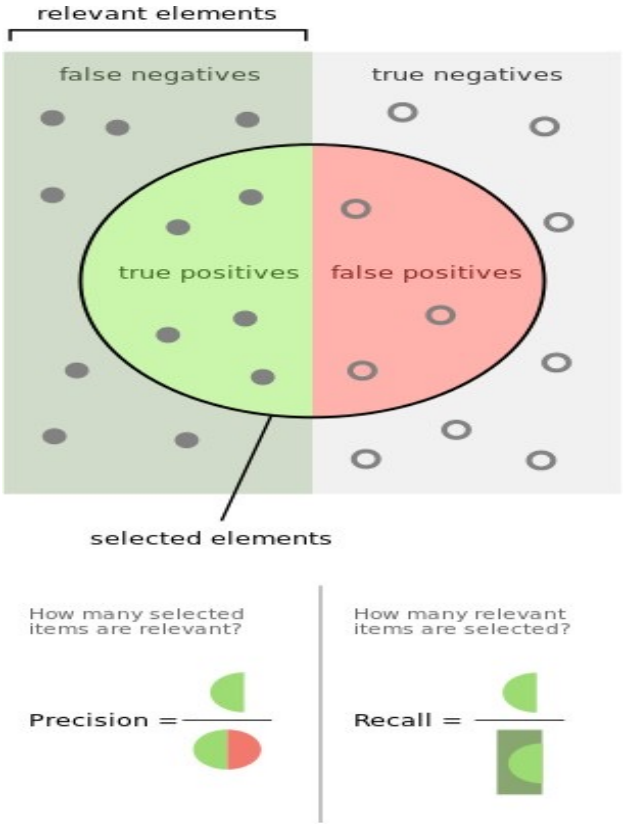
- Precision measures the percentage of items your classifier found that were actually relevant.
- Precision and recall are tied to each other. As one goes up, the other will go down.
- F1 score is a combination of precision and recall.
- F1 score will be low if either precision or recall is low.

Precision, Recall and F1 Score

$$PRE = \frac{TP}{TP + FP}$$

$$REC = TPR = \frac{TP}{P} = \frac{TP}{FN + TP}$$

$$F_1 = 2 \cdot \frac{PRE \cdot REC}{PRE + REC}$$



EIT TE AHO A MĀUI | TePūkenga

Sensitivity and Specificity

$$SEN = TPR = REC = \frac{TP}{P} = \frac{TP}{FN + TP}$$

$$SPC = TNR = \frac{TN}{N} = \frac{TN}{FP + TN}$$

Sensitivity (SEN) measures the recovery rate of the Positives and complimentary, *Specificity (SPC)* measures the recovery rate of the Negatives.

Balanced Accuracy

Predicted Labels

True Labels

	Class 0	Class 1	Class 2
Class 0	T(0,0)		
Class 1		T(1,1)	
Class 2			T(2,2)

Predicted Labels

True Labels

	Class 0	Class 1	Class 2
Class 0	3	0	0
Class 1	7	50	12
Class 2	0	0	18

$$ACC = \frac{T}{n}$$

$$ACC = \frac{3 + 50 + 18}{90} \approx 0.79$$

$$APC\ ACC = \frac{83/90 + 71/90 + 78/90}{3} \approx 0.86$$

EIT TE AHO A MĀUI | TePūkenga

Balanced Accuracy / Average Per-Class Accuracy

Predicted Labels

True Labels	Class 0	Neg Class
Class 0	3	0
Neg Class	7	80

Predicted Labels

True Labels	Class 1	Neg Class
Class 1	50	19
Neg Class	0	21

Predicted Labels

True Labels	Class 2	Neg Class
Class 2	18	0
Neg Class	12	60

Predicted Labels

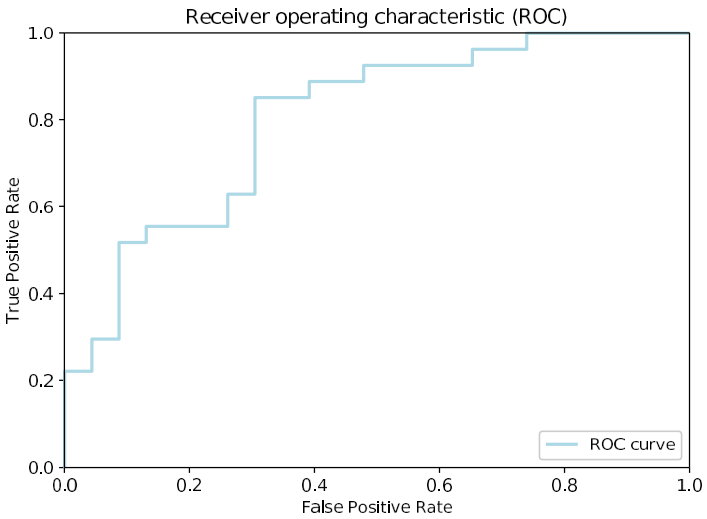
True Labels	Class 0	Class 1	Class 2
Class 0	3	0	0
Class 1	7	50	12
Class 2	0	0	18

$$APC\ ACC = \frac{83/90 + 71/90 + 78/90}{3} \approx 0.86$$

EIT TE AHO A MĀUI | TePūkenga

Receiver Operating Characteristic curve (ROC curve)

- Trade-off between True Positive Rate and False Positive Rate
- ROC can be plotted by changing the prediction threshold



TE AHO A MĀUI
TePūkenga

Extending Binary Metrics to Multi-class Settings

Macro and Micro Averaging

- Micro-averaging is useful if we want to weight each instance or prediction equally, whereas macro-averaging weights all classes equally to evaluate the overall performance of a classifier with regard to the most frequent class labels.

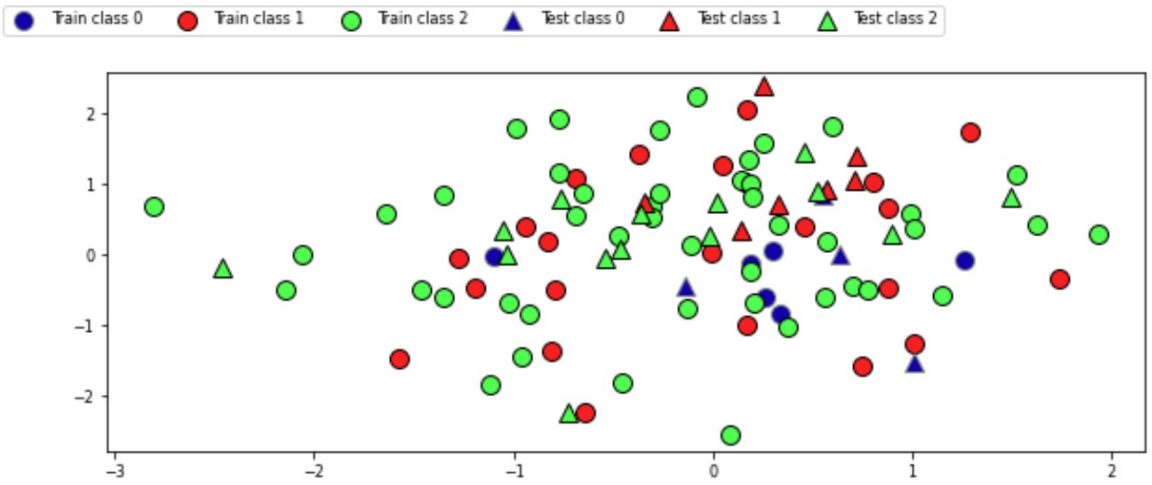
$$PRE_{micro} = \frac{TP_1 + \ldots + TP_c}{TP_1 + \ldots + TP_c + FP_1 + \ldots + FP_c}$$

$$PRE_{macro} = \frac{PRE_1 + \ldots + PRE_c}{c}$$

EIT TE AHO A MĀUI | TePūkenga

Model Selection

- Always evaluate models as if they are predicting future data
- We do not have access to future data, so we pretend that some data is hidden
- Simplest way: the holdout (simple train-test split)
  - Randomly split data (and corresponding labels) into training and test set (e.g. 75%-25%)
  - Train (fit) a model on the training data, score on the test data



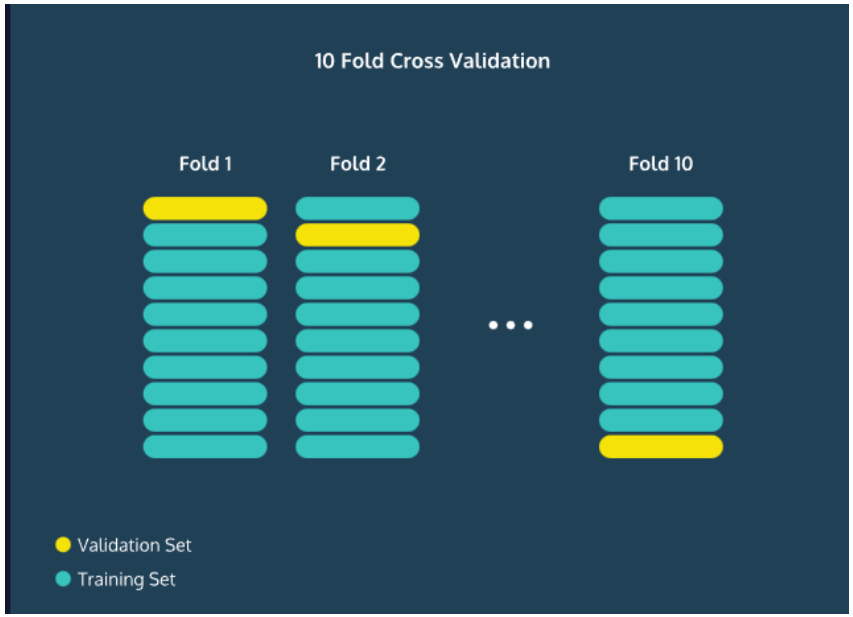
EIT TE AHO A MĀUI

TePūkenga

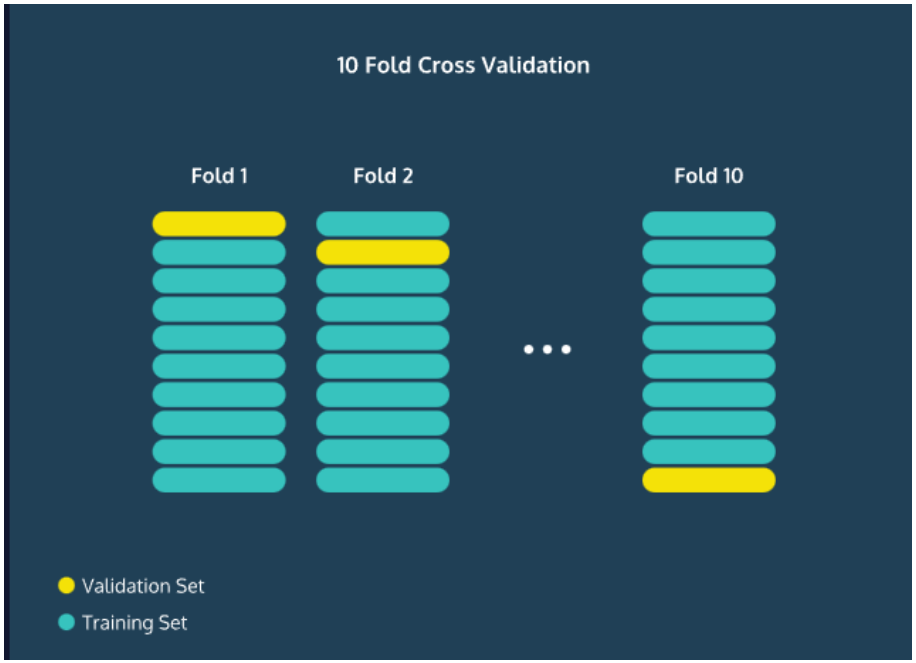
N-Fold Cross-Validation

- Sometimes your dataset is so small, that splitting it 80/20 will still result in a large amount of variance. One solution to this is to perform N-Fold Cross-Validation.
- The central idea here is that we're going to do this entire process N times and average the accuracy.

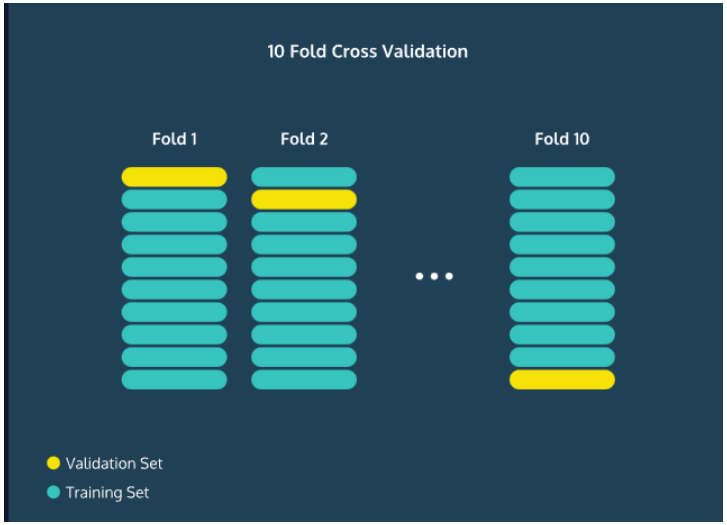
- For example, in 10-fold cross-validation, we'll make the validation set the first 10% of the data and calculate accuracy, precision, recall and F1 score.



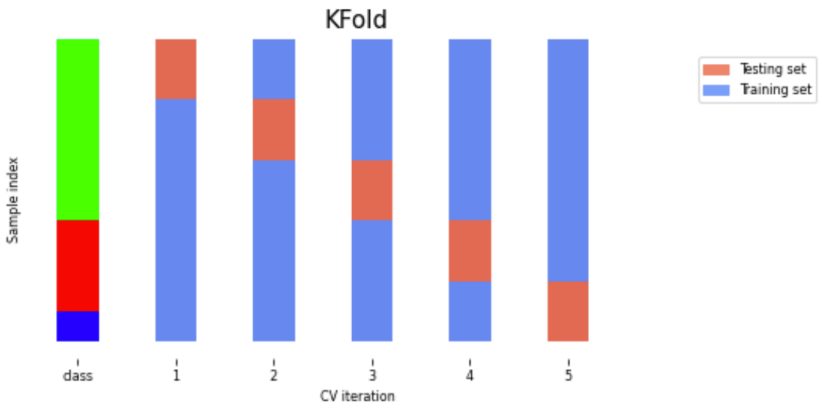
- We'll then make the validation set the second 10% of the data and calculate these statistics again.



- We can do this process 10 times, and every time the validation set will be a different chunk of the data. If we then average all of the accuracies, we will have a better sense of how our model does on average.



K-fold Cross validation



- Each random split can yield very different models (and scores)
  - e.g. all easy (of hard) examples could end up in the test set
- Split data into *k* equal-sized parts, called *folds*
  - Create *k* splits, each time using a different fold as the test set
- Compute *k* evaluation scores, aggregate afterwards (e.g. take the mean)
- Examine the score variance to see how *sensitive* (unstable) models are
- Large *k* gives better estimates (more training data), but is expensive

Overfitting and Regularisation

What is Overfitting?

- If a model is able to represent a particular set of data points effectively but is not able to fit new data well, it is overfitting the data.

Overfitting

- Such a model has one or more of the following attributes:
  - It fits the training data well but performs significantly worse on test data
  - It typically has more parameters than necessary, i.e., it has high model complexity
  - It might be fitting for features that are multi-collinear (i.e., features that are highly negatively or positively correlated)
  - It might be fitting the noise in the data and likely mistaking the noise for features

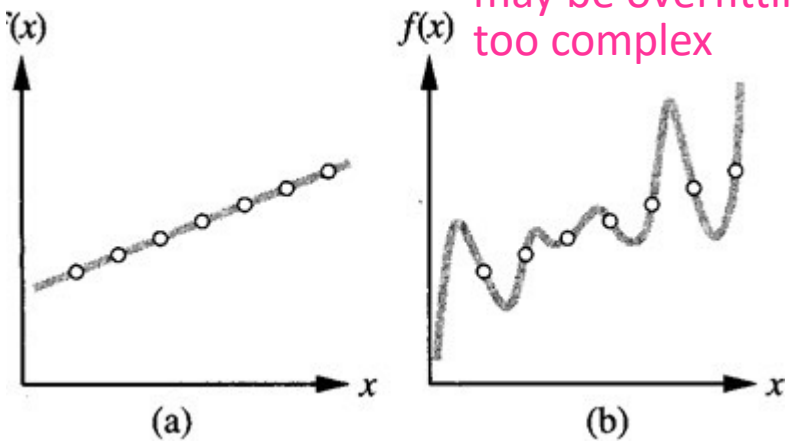
Overfitting

- In practice, we often catch overfitting by comparing the model performance on the training data versus test data. For instance if the R-squared score is high for training data but the model performs poorly on test data, it's a strong indicator of overfitting.

Overfitting

- Overfitting:
  - Small error on the training set but high error on test set (new examples)
  - The classifier has memorised the training examples but has not learned to generalise to new examples!
- It occurs when
  - we fit a model too closely to the particularities of the training set – the resulting model is too specific, works well on the training data but doesn't work well on new data

Ex.1



may be overfitting – too complex

Ex.2

Rule1: may be overfitting – too specific

If age>45, income>100K, has_children=3, divorced=no -> buy_boat=yes

Rule2:

If age>45, income>100K -> buy_boat=yes

EIT TE AHO A MĀUI | TePūkenga

Overfitting (2)

- Various reasons, e.g.
- Issues with the data
  - Noise in the training data
  - Training data does not contain enough representative examples – too small
  - Training data very different than test data – not representative enough
- How the algorithm operates
  - Some algorithms are more susceptible to overfitting than others
  - Different algorithms have different strategies to deal with overfitting, e.g.
    - Decision tree – prune the tree
    - Neural networks – early stopping of the training
    - …

Underfitting

- The model is too simple and doesn't capture all important aspects of the data
  - It performs badly on both training and test data

Rule1: may be overfitting – too specific
If age>45, income>100K, has_children=3,
divorced=no -> buy_boat=yes

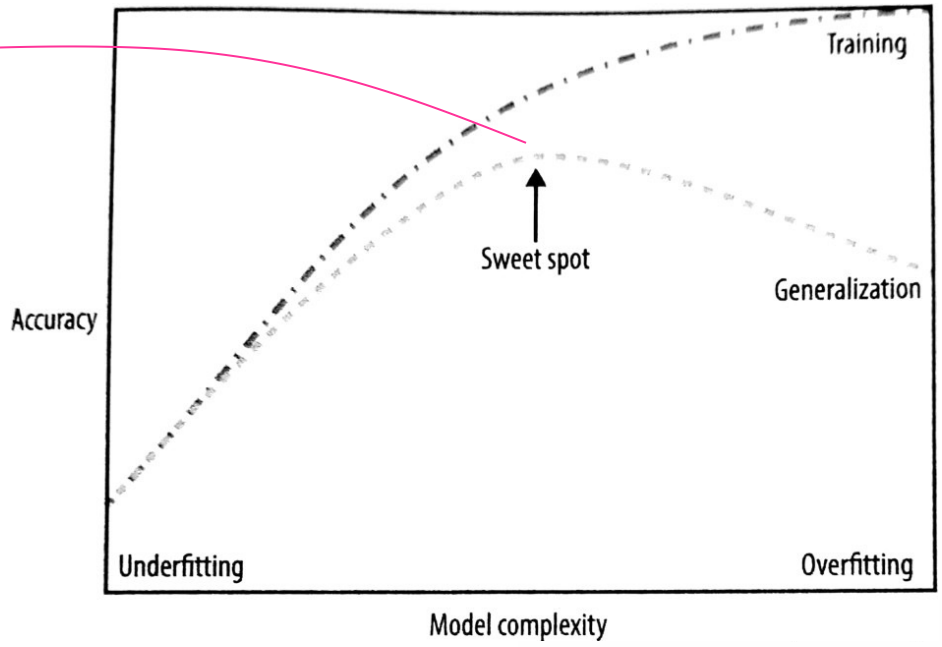
Rule2:
If age>45, income>100K -> buy_boat=yes

Rule3: may be underfitting – too general
If owns_hourse=yes -> buy_boat=yes

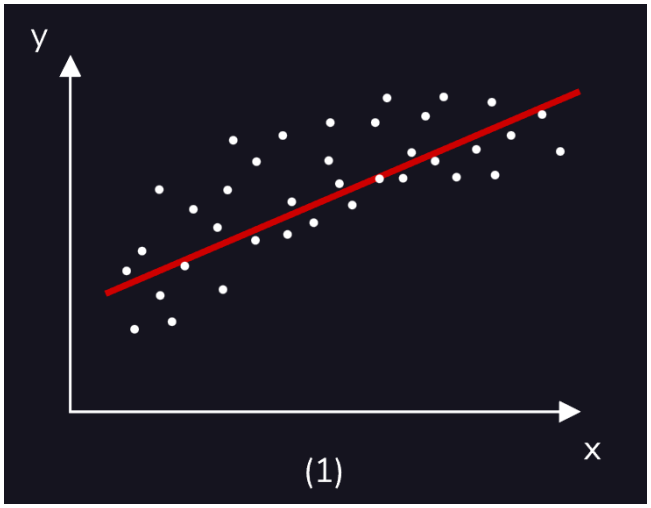
Trade-off between model complexity and generalisation performance

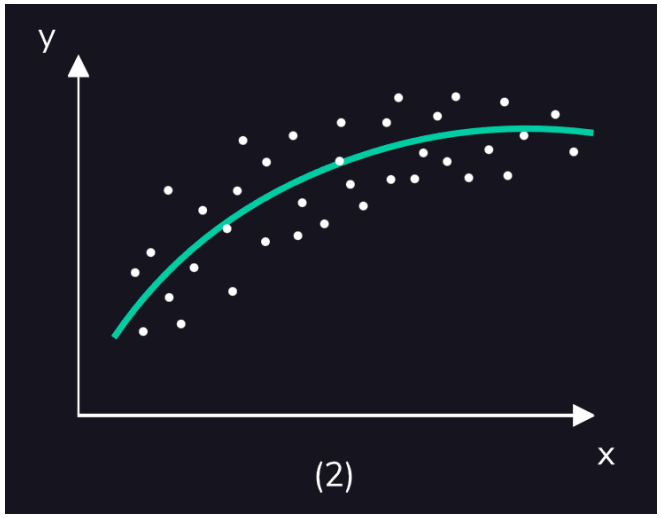
- Generalisation performance = accuracy on test set
- Usually, the more complex we allow the model to be, the better it will predict on the training data
- However, if it becomes to complex, it will start focusing too much on each individual data point, and will not generalise well on new data

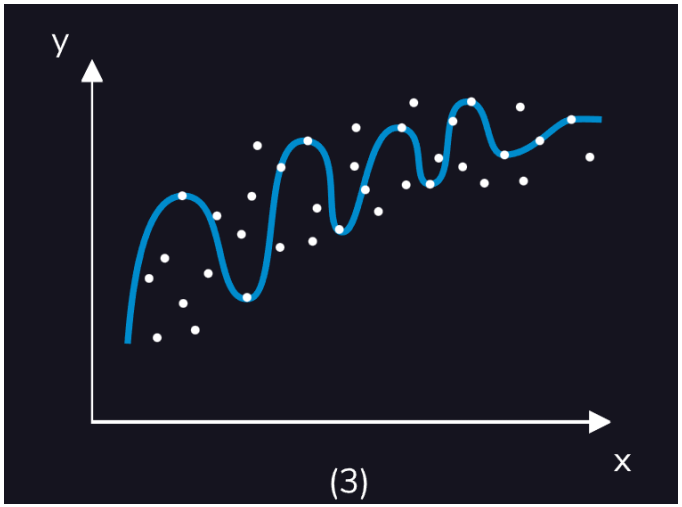
Image from A. Mueller and S. Guido, Introduction to ML with Python



- There is point in between, which will yield the best test accuracy
- This is the model we want to find


y
x
(1)


y
x
(2)


y
x
(3)

The Loss Function

- Let's revisit how the coefficients (or parameters) of a linear regression model are obtained.

$$y = b_0 + b_1 x_1 + b_2 x_2 + e$$

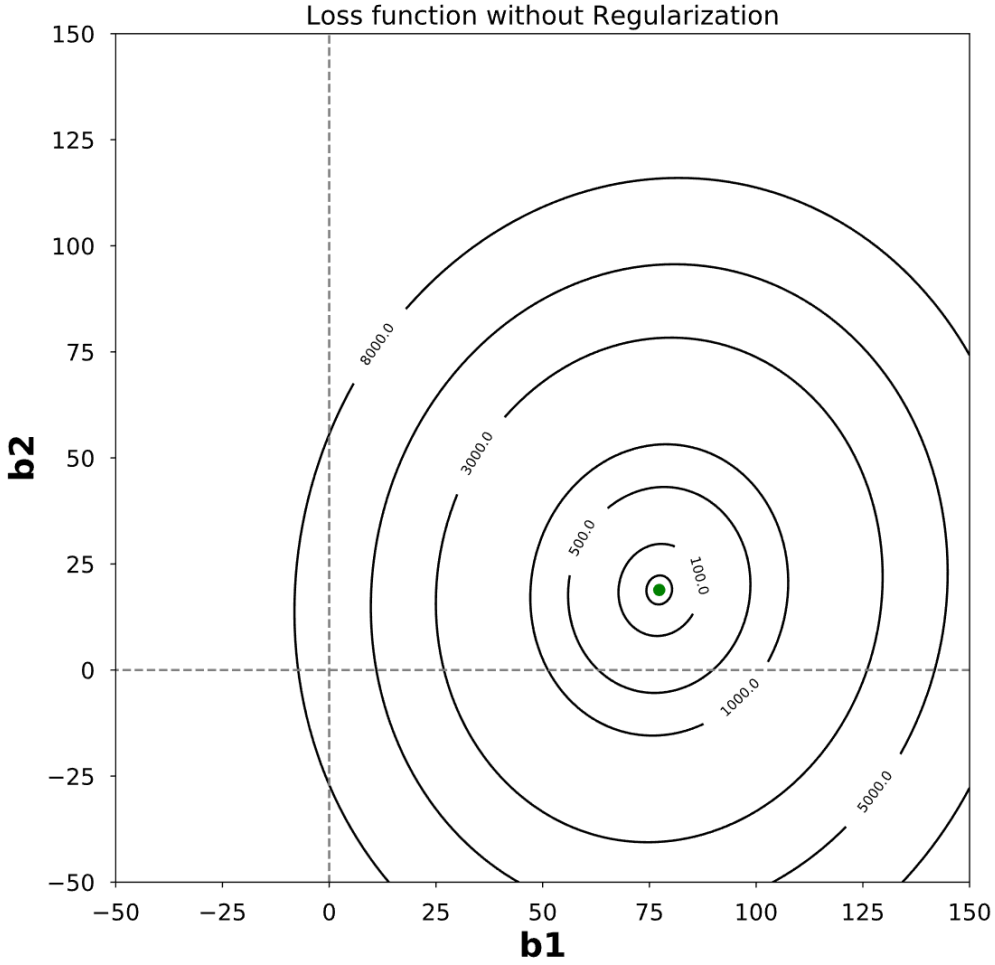
- We solve for the intercept b0 and coefficients b1 and b2 by minimizing the mean squared error. This is also referred to as the loss function (or alternately, as the objective function or cost function) in machine learning:

$$\text{Loss function} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2$$

$$\text{Loss function} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2$$

- The loss function is minimized by the method of Ordinary Least Squares (OLS) and for larger datasets, the gradient descent algorithm is used to arrive at the minimum.

- however, the “best fit” as obtained by minimizing the loss function might not be the fit that generalizes well to new data. Regularization modifies the loss function in a way that might help with this.



$$\text{Loss function} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2$$

Regularisation

- Regularisation means explicitly restricting a model to avoid overfitting
- It is used in some regression models (e.g. Ridge and Lasso regression) and in some neural networks

Why Regularize?

- Regularization plays a very important role in real world implementations of machine learning models.
- It is a statistical technique that minimizes overfitting and is executed during the model fitting step.
- It is also an embedded feature selection method because the parameters of the model are being calculated.

The Regularization Term

- Regularization penalizes models for overfitting by adding a “penalty term” to the loss function. The two most commonly used ways of doing this are known as Lasso (or L1) and Ridge (or L2) regularization.

- Both of these rely on penalizing overfitting by controlling how large the coefficients can get in the first place. The penalty term or regularization term is multiplied by a factor and added to the old loss function as follows:

$$\text{New loss function} = \text{Old loss function} + \alpha * \text{Regularization term}$$

- Remember that the reason we're regularizing is because our model is overfitted to our data (i.e., it is performing well on training data but doesn't generalize well on test data).

- The regularization term is the sum of the absolute values of the coefficients in the case of L1 regularization and the sum of the squares of the coefficients in the case of L2.

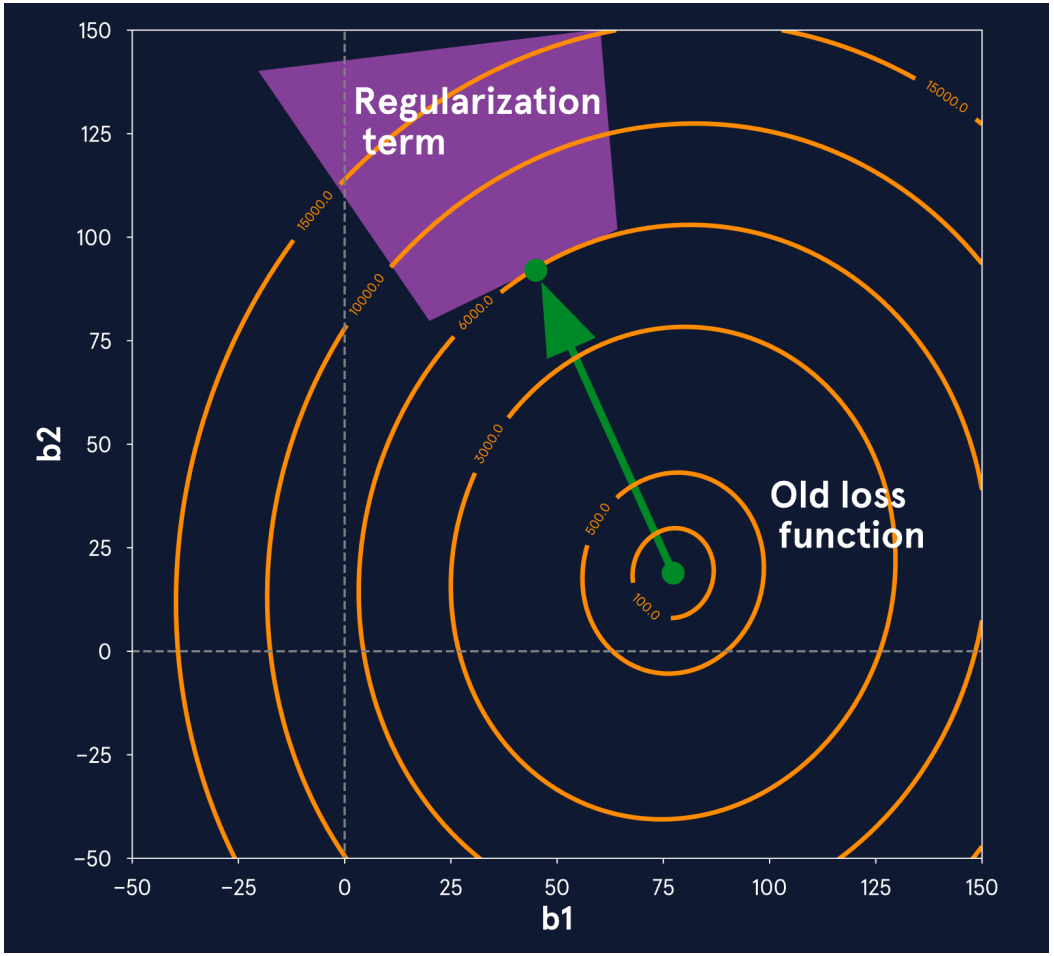
$$\text{L1 Regularization term: } |b_1| + |b_2|$$

$$\text{L2 Regularization term: } b_1^2 + b_2^2$$

- In mathematics, the sum of the magnitudes of a vector is known as its L1 norm (related to “Manhattan distance”) and the square root of the sum of the magnitudes (or the “Euclidean distance” from the origin) is known as its L2 norm - and that is the reason for the names of both methods!

$$\text{L1 Regularization term: } |b_1| + |b_2|$$

$$\text{L2 Regularization term: } b_1^2 + b_2^2$$



Suppose the regularization term constrains the coefficients to values within the pink surface. (Typically, the shape of this surface will depend on the exact mathematical expression for the regularization term. The size of the constraint will be determined by α.)

Ridge and Lasso Regression

Ridge regression

- A regularised version of the standard Linear Regression (LR)
- Also called Tikhonov regularisation
- Uses the same equation as LR to make predictions
- However, the regression coefficients w are chosen so that they not only fit well the training data (as in LR) but also satisfy an additional constraint:
  - the magnitude of the coefficients is as small as possible, i.e. close to 0
- Small values of the coefficients means
  - each feature will have little effect on the outcome
  - small slope of the regression line
- Rationale: a more restricted model (less complex) is less likely to overfit
- Ridge regression uses the so called L2 regularisation (L2 norm of the weight vector)

Ridge regression (2)

- Minimizes the following cost function:

$$\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 + \alpha \sum_{i=1}^{m} w_i^2$$

MSE

regularisation term

n - number of training examples
m – number of regression coefficients (weights)

Goal: high accuracy on training data (low MSE)

low complexity model – w close to 0

- Parameter α controls the trade-off between the performance on the training set and model complexity.

EIT TE AHO A MĀUI | TePūkenga

Ridge regression (3)

$$\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 + \alpha \sum_{i=1}^{m} w_i^2$$

MSE

regularisation term (L2 norm)

- a controls the trade-off between the performance on the training set and model complexity
  - Increasing α makes the coefficients smaller (close to 0); this typically decreases the performance on the training set but may improve the performance on the test set
  - Decreasing α means less restricted coefficients. For very small α , Ridge Regression will behave similarly to LR

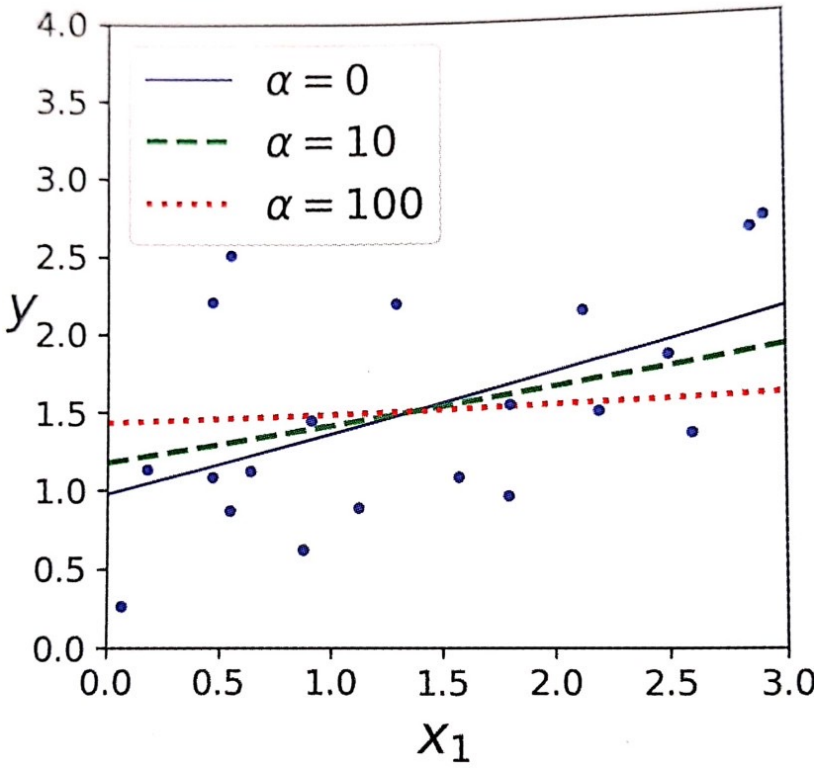


Image from A. Geron, Hands-on ML with Scikit-learn, Keras & TensorFlow

TE AHO A MĀUI

TePūkenga

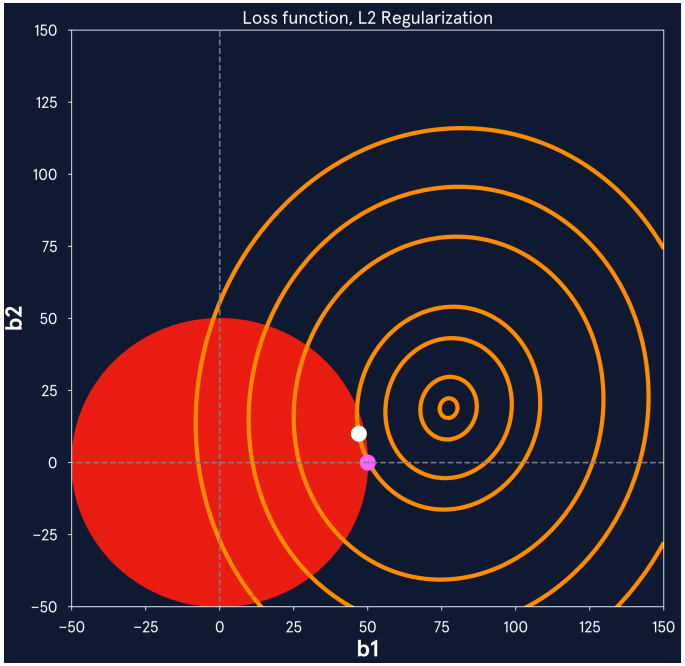
L2 or Ridge Regularization

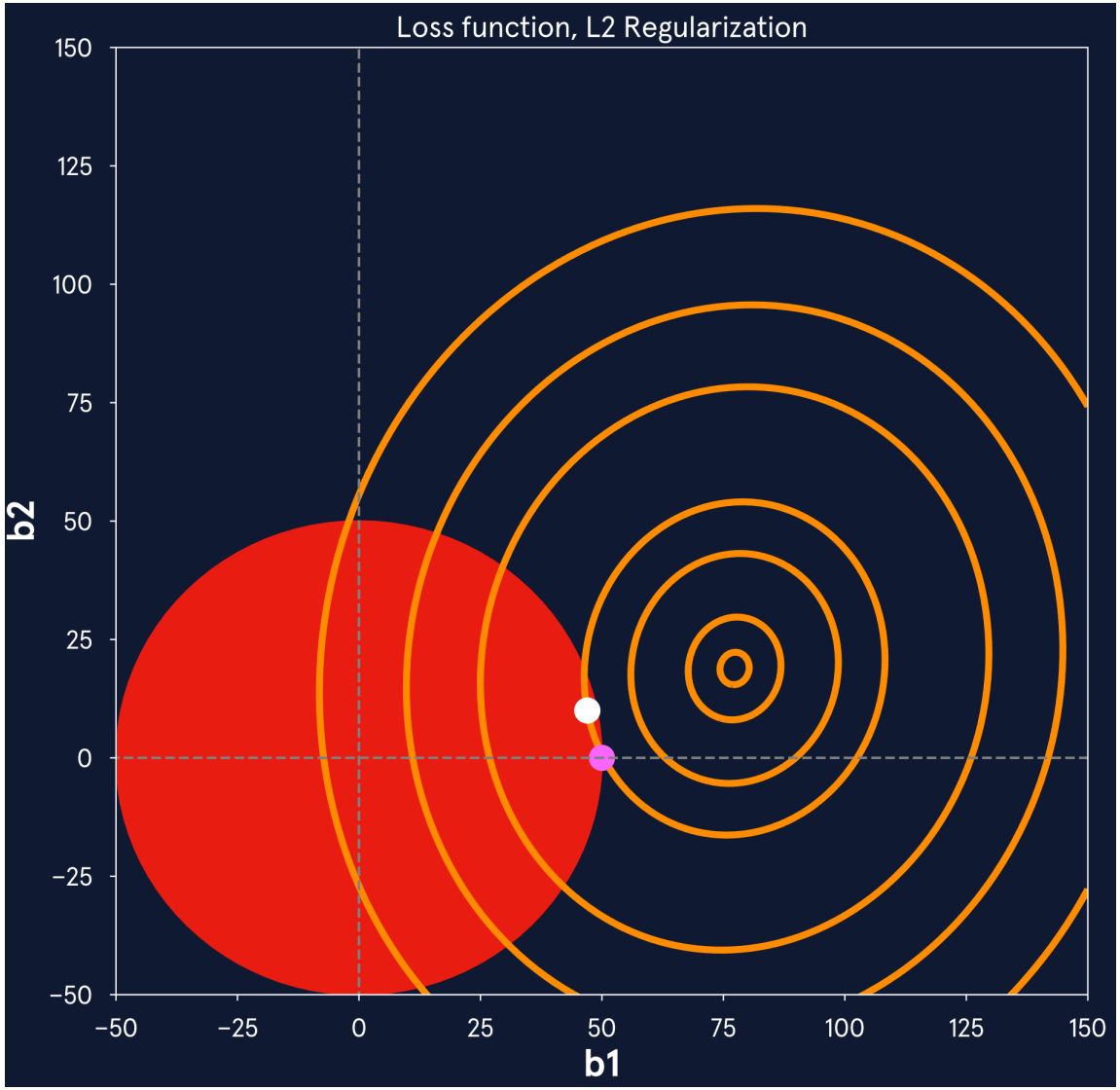
- L2 regularization is also called “Ridge” regularization as the mathematical formulation of it belongs to a class of analysis methods known as “ridge analysis”. The modified loss function in the case of L2 regularization looks as follows:

$$\text{L2 Loss} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha * (b_1^2 + b_2^2)$$

- this would mean constraining the regularization term to the following surface:

$$b_1^2 + b_2^2 \leq s^2$$




Loss function, L2 Regularization
150
125
100
75
50
25
0
−25
−50
b2
−50
−25
0
25
50
75
100
125
150
b1

Lasso regression

- Another regularised version of the standard LR
- LASSO = Least Absolute Shrinkage and Selection Operator Regression
- As Ridge Regression, it adds a regularisation term to the cost function but it uses the L1 norm of the regression coefficient vector w

$$\underbrace{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}_{\text{MSE}} + \underbrace{\alpha \sum_{i=1}^{m} |w_i|}_{\text{regularisation term (L1 norm)}}$$

Goal: high accuracy on training data (low MSE)

low complexity model

- Consequence of using L1 – some w will become exactly 0
- => some features will be completely ignored by the model – a form of automatic feature selection
- Less features – simpler model, easier to interpret

EIT TE AHO A MĀUI | TePūkenga

Lasso regression (2)

$$\underbrace{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}_{\text{MSE}} + \underbrace{\alpha \sum_{i=1}^{m} |w_i|}_{\text{regularisation term (L1 norm)}}$$

- As in Ridge Regression:
  - α controls the trade-off between the performance on the training set and model complexity
  - Increasing/decreasing α - similar reasoning as before

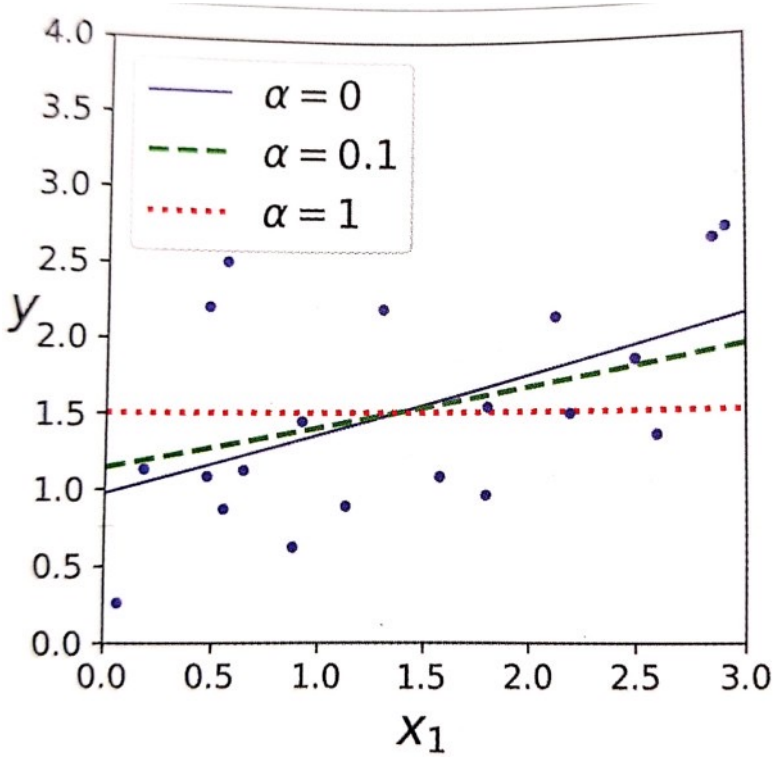


Image from A. Geron, Hands-on ML with Scikit-learn, Keras & TensorFlow

L1 or Lasso Regularization

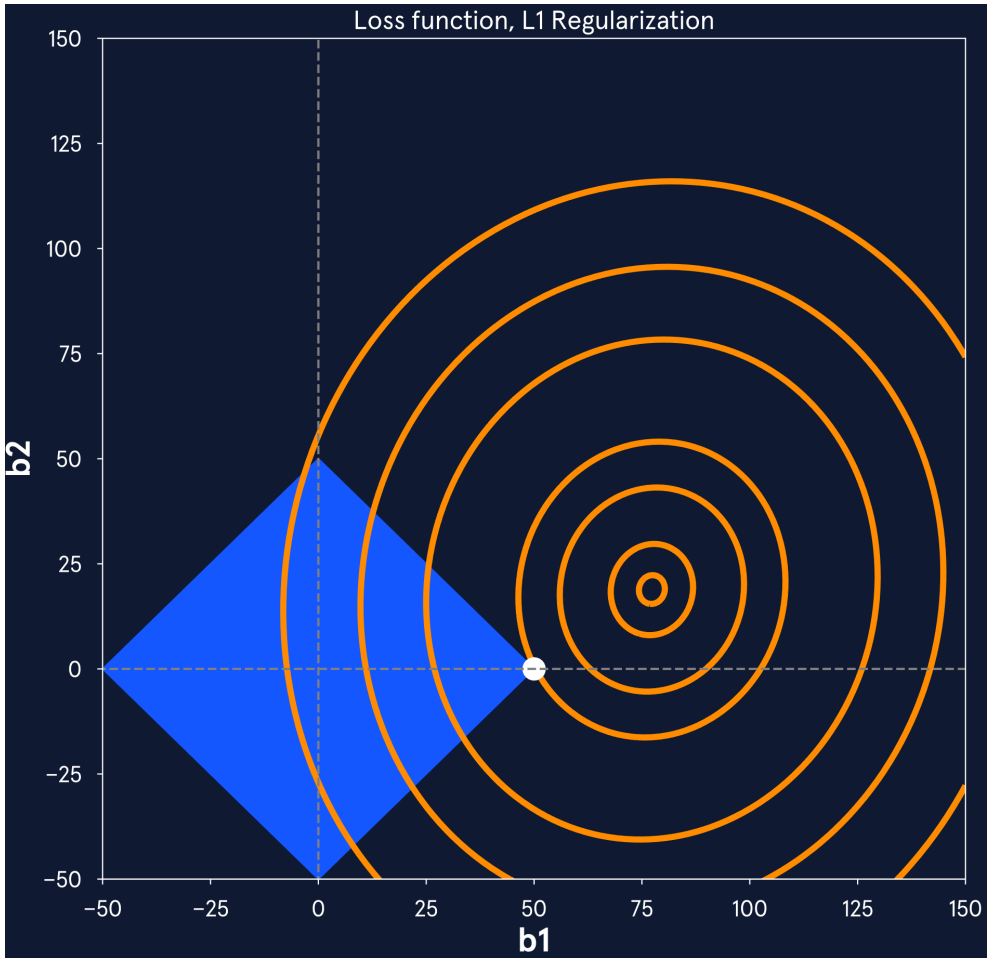
- In the case of the two-feature linear regression scenario we were looking at in the previous exercise, our new loss function for L1 regularization looks as follows:

$$\text{L1 Loss} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha * (|b_1| + |b_2|)$$

L1 or Lasso Regularization

- Minimizing this new loss function essentially means restricting the size of the regularization term while minimizing the old loss function. Let's say that our regularization term can take a maximum value,

$$|b_1| + |b_2| \leq s$$



- The value of (b1,b2) that satisfies the regularization constraint while minimizing the loss function is denoted by the white dot.

Hyperparameter Tuning

- `alpha` is what is known as a hyperparameter in machine learning. It is not learned during the model fitting step the way model parameters are. Rather, it's chosen prior to fitting the model and is used to control the learning process.

$$\text{L1 Loss} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha * (|b_1| + |b_2|)$$

$$\text{L2 Loss} = \frac{1}{n}\sum_{i=1}^{n}(y_i - b_0 - b_1 x_{1i} - b_2 x_{2i})^2 + \alpha * (b_1^2 + b_2^2)$$

- To avoid either of these extreme scenarios, we need to find the right amount of regularization by tuning and this process is known as hyperparameter tuning.

	Lasso Regularization	Ridge Regularization
Regularization / Penalty term	L1 norm (sum of absolute values) of coefficients	L2 norm (sum of squared values) of coefficients
Constraint surface	Diamond shaped in 2D	Circle shaped in 2D
Coefficients after Regularization	All the coefficients shrink in their magnitude and some can be zero.	All the coefficients shrink in their magnitude but never approach zero.
Feature Selection Method	Yes! Features corresponding to the zero value coefficients are eliminated.	No! But it helps in emphasizing the relative feature importance.

Summary

- Overfitting and regularisation
  - Overfitting - high accuracy on training data but low accuracy on test data (low generalisation)
  - High model complexity -> low generalisation
  - Regularisation is a method to avoid overfitting – it makes the model more restrictive (less complex)
  - Ridge and Lasso regression are regularised linear regression models

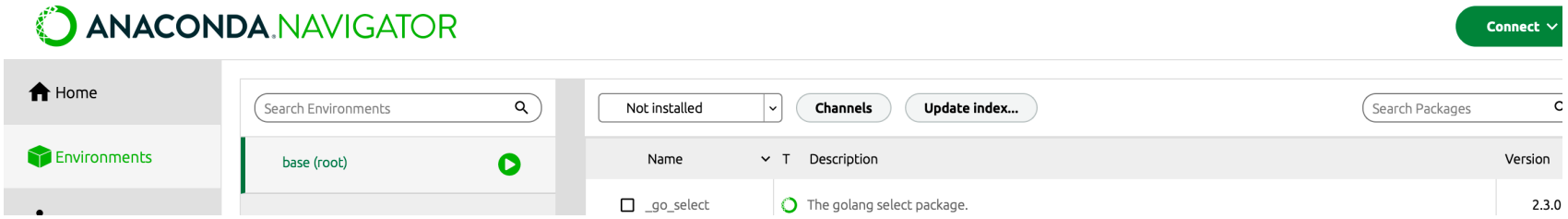
LABS This Week

- Lab 1: Building a Linear Regression model with multiple input variables using a real-world dataset.
- Lab 2: Building a Polynomial Regression model using a real-world dataset

QUANDL

- The data is gathered from Quandl.
- Quandl is a platform for financial and stocks related data.
- This is just for learning purpose, can't be used for any finacial analysis.
- Company we have taken is Apple Inc.
- Feature generation is used to create features from existing features.

Install Quandl



Use Qunadl

- import quandl as qndl
- df = qndl.get('WIKI/AAPL') OR

First you need to register yourself on this website and get your personal API-KEY to access data from QUANDL databases. For detailed documentation, please see URL:

https://docs.quandl.com

```
import pandas as pd
import quandl as qndl
# Symbol of the stock that we want for analysis
symbol = "WIKI/AAPL"
qndl.ApiConfig.api_key = "####################" # you will get your KEY
df = qndl.get(symbol)
print(df.info())
```

Date	Open	High	Low	Close	Volume	Ex-Dividend	Split Ratio	Adj. Open	Adj. High	Adj. Low	Adj. Close	Adj. Volume
1980-12-12	28.75	28.87	28.75	28.75	2093900.0	0.0	1.0	0.422706	0.424470	0.422706	0.422706	117258400.0
1980-12-15	27.38	27.38	27.25	27.25	785200.0	0.0	1.0	0.402563	0.402563	0.400652	0.400652	43971200.0
1980-12-16	25.37	25.37	25.25	25.25	472000.0	0.0	1.0	0.373010	0.373010	0.371246	0.371246	26432000.0
1980-12-17	25.87	26.00	25.87	25.87	385900.0	0.0	1.0	0.380362	0.382273	0.380362	0.380362	21610400.0
1980-12-18	26.63	26.75	26.63	26.63	327900.0	0.0	1.0	0.391536	0.393300	0.391536	0.391536	18362400.0

- From this data, we can develop a linear regression model to predict the future values for APPLE’s stocks.
- Starting from the data frame, you need to perform the linear regression analysis and predict the stock values of a company. You can select any company for this analysis, and complete and answer the following actions:

Actions	Details
Action I. **(3 marks)**	Give an initial analysis of the data frame by explaining different columns. During this analysis, you need to identify the **FEATURES (X)** that we can use to predict the **response variable/target (y)** that is "**CLOSE"** (column) value for the stock on a specific date.
Action II. **(2 marks)**	Separate the data into **Features (X)** and **response variable (y)**.
Action III. **(2 marks)**	Use a scatter-plot to plot the **OPEN** and **CLOSE** values.

- refer to topic 2-lab 2(step 1-3)

Move the data ahead

- forecast= 30
- df[‘your target'] = df[forecast_col].shift(-forecast)
- df.dropna(inplace=True)

Action IV. **(4 marks)**	Find the correlations between each identified feature with **CLOSE** and determine that these are positive or negative.

- df.corr().loc[‘xxxxxxx'].plot(kind='barh', figsize=(8,2))

Action V. **(4 marks)**	Split the data into two parts in a ratio of 0.7 for training and 0.3 for testing.
Action VI. **(5 marks)**	Fit a multivariate linear regression model on the training data using all selected features.

- Refer to topic 3 - lab 1(step3-4)

Action VII. **(6 marks)**	What are the intercept (θ_0) and coefficients ($\theta_1, \theta_2, \ldots, \theta_n$) of the model?
Action VIII. **(4 marks)**	What is the R^2 score for the training data and for the test data?

- Refer to topic 2 – lab 2 (step 5 and 7)

Action IX. **(4 marks)**	Create sets of imaginary values for the FEATURES, at least 3 values sets, and predict the CLOSE value for these imaginary FEATURES values. Report your predicted values.

- Refer to topic 1 – lab 2 (step 6)

Action X. **(6 marks)**	Re-instantiate your linear regression model with the parameter "fit_intercept" set to "FALSE" and rerun you analysis on the entire features data (X). What are coefficients $(\theta_1, \theta_2, \ldots, \theta_n)$ of the model?

- Create a new model and apply LinearRegression(fit_intercept = False)

- Deadline should be Oct 8/ 24:00(time in China)
- NOT

Due date	Monday, 9 October 2023, 5:00 AM

- It’s good to find some resources to learn. But do not copy others’ work, especially add some tasks not mentioned in the file.